МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ»
ІНСТИТУТ ПРИКЛАДНОГО СИСТЕМНОГО АНАЛІЗУ

Лабораторна робота №2
з дисципліни “Теорія інформації і кодування”
на тему: “ Стиснення даних в дискретних повідомленнях з використанням методу Шеннона-Фано.”

Виконав:
студент III курсу
групи ДА-11
Сидько Нікіта Сергійович

Київ-2023

Мета: ознайомлення з методом ефективного кодування даних без втрат Шеннона-Фано, розробка алгоритму та програми для його реалізації, дослідження ефективності кодування даних у файлах різних типів і довільної довжини.

Завдання

1.Ознайомитися з методикою ефективного кодування Шеннона-Фано.

2. Використовуючи табличний процесор MS Excel закодуйте методом Шеннона-Фано текстове повідомлення з вашим прізвищем ім'ям, по батькові, датою і місцем народження. Підрахуйте ентропію текстового повідомлення, середню довжину кодових комбінацій закодованого повідомлення, коефіцієнт стиснення і оцініть ефективність кодування. Перевірте префіксність коду і розкодуйте повідомлення.

3. Розробити програму на мові програмування високого рівня для кодування і декодування текстових файлів довільної довжини. При вводі текстових даних передбачити два режиму вводу кодів: введення з клавіатури, а також введення даних з текстового файлу з використанням однобайтових кодових таблиць. За допомогою розробленої програми перевірити дані, отримані при виконанні п.2.

4. За допомогою розробленої при виконанні п.3 програми дослідити залежність коефіцієнту стиснення від довжини текстового файлу. Результати представити в табличній формі.

5. Модіфікувати програму, розроблену при виконанні п.4, для стиснення файлів довільних типів даних за алгоритмом Шеннона-Фано. Провести дослідження ефективності розробленої програми шляхом порівняння результатів стиснення текстових, мультимедійних і exe –файлів з допомогою розробленої програми і архіватора WinZip . Результати занести в таблицю

Хід роботи

Запишемо повідомлення та випишемо його статистику

сидько нікіта сергійович 25.05.2004 київ

	Символ	Частота	Ймовірність	Log(P[i])	Ентропія
	і	3	0,075	-3,73697	0,28027
	ї	1	0,025	-5,32193	0,13305
	а	1	0,025	-5,32193	0,13305
	в	2	0,05	-4,32193	0,2161
	г	1	0,025	-5,32193	0,13305
	д	1	0,025	-5,32193	0,13305
	е	1	0,025	-5,32193	0,13305
	и	3	0,075	-3,73697	0,28027
	й	1	0,025	-5,32193	0,13305
	к	3	0,075	-3,73697	0,28027
	н	1	0,025	-5,32193	0,13305
	о	2	0,05	-4,32193	0,2161
	р	1	0,025	-5,32193	0,13305
	с	2	0,05	-4,32193	0,2161
	т	1	0,025	-5,32193	0,13305
	ч	1	0,025	-5,32193	0,13305
	ь	1	0,025	-5,32193	0,13305
	пробіл	4	0,1	-3,32193	0,33219
	.	2	0,05	-4,32193	0,2161
	0	3	0,075	-3,73697	0,28027
	2	2	0,05	-4,32193	0,2161
	4	1	0,025	-5,32193	0,13305
	5	2	0,05	-4,32193	0,2161
Сума	23	40	1	-108,064	4,34644

Amount of info :	173,8575738
Max Entropy:	4,523561956
Entropy:	4,3463934
Absolute excess:	0,177168556
Relative excess:	0,039165719

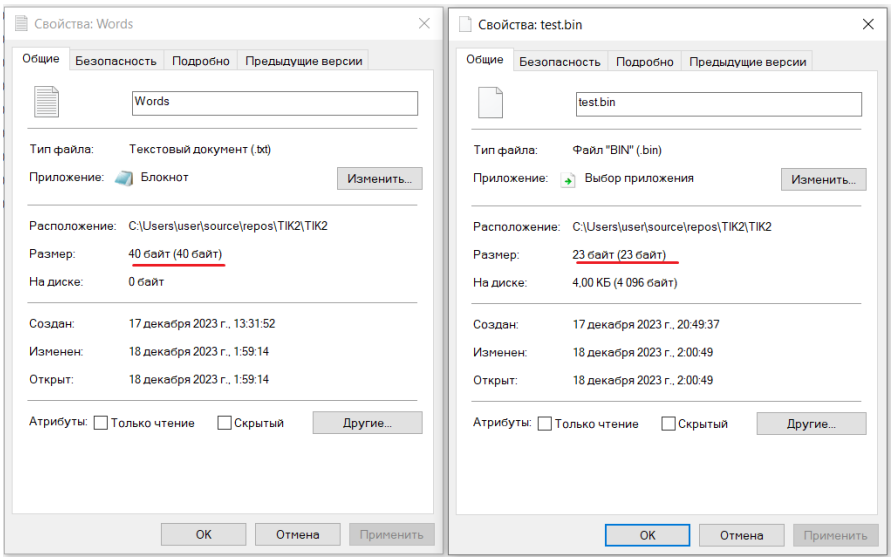
Символ	Частота	1	2	3	4	5	6	
пробіл	4	0	0	0	0			Код : 0000
і	3	0	0	0	1			Код : 0001
и	3	0	0	1				Код : 001
к	3	0	1	0	0			Код : 0100
о	3	0	1	0	1			Код : 0101
в	2	0	1	1	0			Код : 0110
о	2	0	1	1	1			Код : 0111
с	2	1	0	0	0	0		Код : 10000
.	2	1	0	0	0	1		Код : 10001
2	2	1	0	0	1			Код : 1001
5	2	1	0	1	0			Код : 1010
ї	1	1	0	1	1	0		Код : 10110
а	1	1	0	1	1	1		Код : 10111
г	1	1	1	0	0	0		Код : 11000
д	1	1	1	0	0	1		Код : 11001
е	1	1	1	0	1	0		Код : 11010
й	1	1	1	0	1	1	0	Код : 110110
н	1	1	1	0	1	1	1	Код : 110111
р	1	1	1	1	0	0		Код : 11100
т	1	1	1	1	0	1		Код : 11101
ч	1	1	1	1	1	0		Код : 11110
ь	1	1	1	1	1	1	0	Код : 111110
4	1	1	1	1	1	1	1	Код : 111111

Зробимо перевірку кодом:

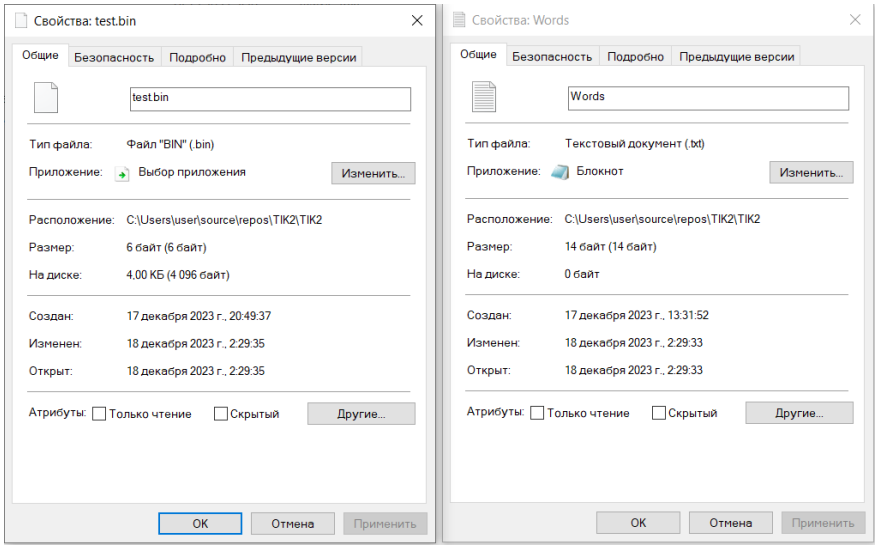
```
Шеннон-Фано дерево:
Символ  Код
        0000
і       0001
и       001
к       0100
0       0101
в       0110
о       0111
с       10000
.       10001
2       1001
5       1010
і       10110
а       10111
г       11000
д       11001
е       11010
й       110110
н       110111
р       11100
т       11101
ч       11110
ь       111110
4       111111
```

Бачимо, що значення співпали.

```
1000000111001111110010001110000110111000101000001111011011100001000011010111001100000011101100111011000111110000010011010
0100010101101010001100101010101111111000001000011011001100000000
Decoded Sentence: сидько нікіта сергійович 25.05.2004 київ
```



Коеф стиснення = 1.73



Коеф стиснення = 2.3

Пропишемо в коді, наприклад для File.xlsx. Та протестуємо чи буде відбуватися стиснення файлу:

```cpp
int totalCharacters = 0;
std::map<char, int> charCount = readCharacterCounts("123123.xlsx", totalCharacters);

std::vector<Symbol> probabilityVector;

calculateValues(charCount, totalCharacters);
sortTable(charCount, probabilityVector, totalCharacters);
printTable(probabilityVector, charCount);

Node* root = nullptr;
buildShannonFanoTree(probabilityVector, root, 0, probabilityVector.size() - 1, "");

std::cout << "\nШеннон-Фано дерево:" << std::endl;
std::cout << "Символ\tКод" << std::endl;

traverseAndPrintTree(root, "");

std::map<char, std::string> encodingMap;
std::map<std::string, char> decodingMap;

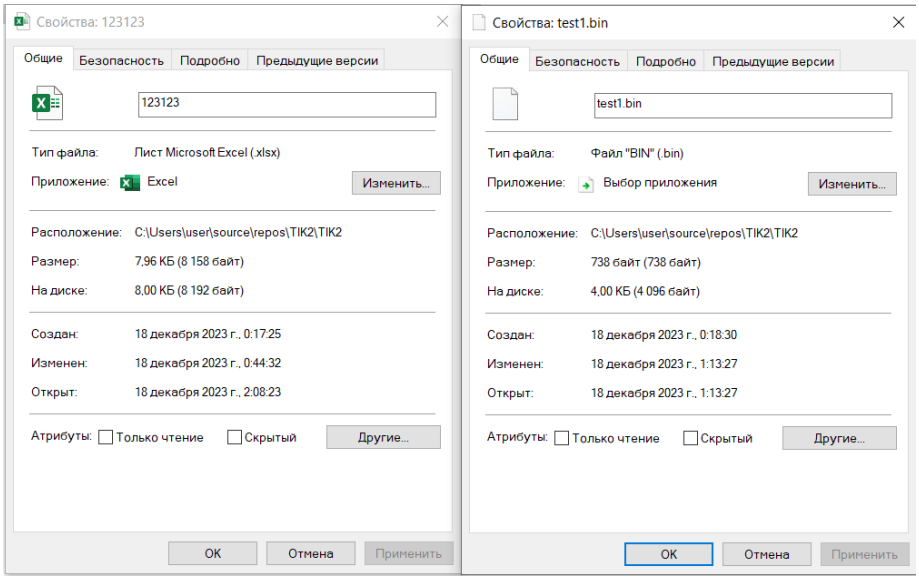
for (const Symbol& element : probabilityVector) {
    encodingMap[element.symbol] = element.code;
}

for (const Symbol& element : probabilityVector) {
    decodingMap[element.code] = element.symbol;
}

std::string sentence = readSentenceFromFile("123123.xlsx");
std::string encodedString = encodeSentence(sentence, encodingMap);
writeBinaryFile("test.bin", encodedString);

decodeBinaryFile("test.bin", decodingMap);

delete root;
```



Вихідний файл	Програма	Архіватор
Назва: Words1.txt **Розмір**: 280	**Розмір:** 162 **Коеф :** 1.72	**Розмір:** 199 **Коеф :** 1.4
Назва: Words1.txt **Розмір**: 534	**Розмір**: 320 **Коеф:** 1.66	**Розмір:** 348 **Коеф:** 1.53

Висновок.

У ході виконання 2гої лабораторної роботи, була розроблена програма, для стиснення файлу методом шеннона-фано. Для прикладу був взятий текстовий документ у якому було написано ПІБ, текст був закодований на

записаний у бітах в BIN файл, після чого файл був побітово зчитаний та розкодований

Лістинг коду :

```
#include <iostream>
#include <fstream>
#include <map>
#include <cmath>
#include <vector>
#include <algorithm>
#include <string>
#include "Windows.h"
#include <bitset>
#include <sstream>


void setConsoleEncoding() {

    SetConsoleCP(1251);
    SetConsoleOutputCP(1251);

}

struct Symbol {
    char symbol;
    int frequency;
    std::string code;
};

std::map<char, int> readCharacterCounts(const std::string& filename, int&
totalCharacters) {

    std::map<char, int> charCount;

    uint8_t character;

    std::ifstream inputFile(filename, std::ios::binary);

    if (!inputFile.is_open()) {
        std::cerr << "Unable to open file." << std::endl;
        exit(1);
    }

    while (true) {
        character = inputFile.get();
        if (inputFile.eof())
        {
            break;
        }
        charCount[character]++;
        totalCharacters++;
    }

    inputFile.close();

    return charCount;
}

bool compareProbability(const Symbol& a, const Symbol& b) {
    return a.frequency > b.frequency;
}

void sortTable(const std::map<char, int>& charCount, std::vector<Symbol>&
probabilityVector, int totalCharacters) {
```

```
    for (const auto& pair : charCount) {
        float probability = static_cast<float>(pair.second) / totalCharacters;
        probabilityVector.push_back({ pair.first, pair.second, "" });
    }

    std::sort(probabilityVector.begin(), probabilityVector.end(),
compareProbability);
}

void printTable(const std::vector<Symbol>& probabilityVector, const
std::map<char, int>& charCount) {
    std::cout << "Відсортована таблиця:" << std::endl;
    std::cout << "Символ\tЧастота\tКод" << std::endl;

    for (const auto& symbol : probabilityVector) {
        if (symbol.symbol == ' ') {
            std::cout << "Space\t";
        }
        else {
            std::cout << symbol.symbol << "\t";
        }
        std::cout << symbol.frequency << "\t" << symbol.code << std::endl;
    }
}

void calculateValues(const std::map<char, int>& charCount, int totalCharacters) {
    float PiSUM = 0;
    float LogSUM = 0;
    float EntrSUM = 0;

    std::cout << "Таблиця:" << std::endl;
    std::cout << "Символ\tЧастота\tP[i]\t\tLog(P[i])\tEntropy" << std::endl;

    for (auto& pair : charCount) {
        PiSUM += static_cast<float>(pair.second) / totalCharacters;
        LogSUM += log2(static_cast<float>(pair.second) / totalCharacters);
        EntrSUM += static_cast<float>(pair.second) / totalCharacters *
log2(static_cast<float>(pair.second) / totalCharacters);

        if (pair.first == ' ') {
            std::cout << "Space\t";
        }
        else {
            std::cout << pair.first << "\t";
        }

        std::cout << pair.second << "\t" << static_cast<float>(pair.second) /
totalCharacters
            << "\t\t" << log2(static_cast<float>(pair.second) / totalCharacters)
            << "\t" << -1 * static_cast<float>(pair.second) / totalCharacters *
log2(static_cast<float>(pair.second) / totalCharacters)
            << std::endl;
    }

    EntrSUM *= -1;

    std::cout << std::endl;
    std::cout << "Symbol count: " << totalCharacters << std::endl;
    std::cout << "Sum P[i]: " << PiSUM << std::endl;
    std::cout << "Sum Log(P[i]): " << LogSUM << std::endl;
    std::cout << "H: " << EntrSUM << std::endl;
    std::cout << std::endl;

    std::cout << "Amount of info: " << totalCharacters * EntrSUM << std::endl;
    std::cout << "Max entropy: " << log2(charCount.size()) << std::endl;
```

```
    std::cout << "Absolute excess: " << log2(charCount.size()) - EntrSUM <<
std::endl;
    std::cout << "Relative excess: " << (log2(charCount.size()) - EntrSUM) /
EntrSUM << std::endl;
    std::cout << std::endl;
}

struct Node {

    Symbol symbol;
    Node* left;
    Node* right;
};

void traverseAndPrintTree(Node* root, std::string currentCode) {
    if (root != nullptr) {
        if (root->left != nullptr) {
            traverseAndPrintTree(root->left, currentCode + "0");
        }

        if (root->right != nullptr) {
            if (root->left == nullptr) {
                std::cout << root->symbol.symbol << "\t" << currentCode <<
std::endl;
            }
            traverseAndPrintTree(root->right, currentCode + "1");
        }

        // Добавлен блок обработки листового узла
        if (root->left == nullptr && root->right == nullptr) {
            std::cout << root->symbol.symbol << "\t" << currentCode << std::endl;
        }
    }
}

void buildShannonFanoTree(std::vector<Symbol>& probabilityVector, Node*& root,
int start, int end, std::string currentCode) {

   // std::cout << "1" << std::endl;

    if (start > end) {
        return;
    }

    if (start == end) {
        root = new Node;
        root->symbol = probabilityVector[start];
        root->left = nullptr;
        root->right = nullptr;
        probabilityVector[start].code += currentCode;
        return;
    }

    int sum = 0;
    for (int i = start; i <= end; ++i) {
        sum += probabilityVector[i].frequency;
    }

    int currentSum = 0;
    int index = start;
    while (currentSum < sum / 2)
    {
        currentSum += probabilityVector[index].frequency;
        index++;
    }
```

```
    /*
    while ( currentSum + probabilityVector[index].frequency * 2 <= sum) {
        currentSum += probabilityVector[index].frequency;

        // Проверка, ближе ли текущий индекс к середине
        int midpointDifference = std::abs(2 * currentSum - sum);
        if (midpointDifference < closestMidpoint) {
            closestMidpoint = midpointDifference;
            index++;
        }
        else {
            break;
        }
    }
    */
    root = new Node;
    root->left = nullptr;
    root->right = nullptr;

    buildShannonFanoTree (probabilityVector, root->left, start, index - 1,
currentCode + "0");
    buildShannonFanoTree (probabilityVector, root->right, index, end, currentCode
+ "1");
}

std::string readSentenceFromFile(const std::string& filename) {

    std::ifstream file(filename);

    if (!file.is_open()) {
        std::cerr << "Unable to open file: " << filename << std::endl;
        exit(1);
    }

    std::string sentence;
    std::getline(file, sentence);

    file.close();

    return sentence;
}

std::string encodeSentence(const std::string& sentence, const std::map<char,
std::string>& encodingMap) {
    std::string encodedString;

    for (char character : sentence) {
        auto it = encodingMap.find(character);
        if (it != encodingMap.end()) {
            encodedString += it->second;
        }
        else {
            std::cerr << "Symbol not found: " << character << std::endl;
        }
    }
    return encodedString;
}

void writeBinaryFile(const std::string& filename, const std::string& data) {

    std::ofstream file(filename, std::ios::binary);
    if (file.is_open()) {
        // Конвертуємо рядок бітів у відповідний байтовий вигляд
        for (std::size_t i = 0; i < data.length(); i += 8) {
```

```
            std::bitset<8> byte;
            for (std::size_t j = 0; j < 8; ++j) {
                if (i + j < data.length()) {
                    byte[7 - j] = (data[i + j] == '1');
                }
            }
            char byteChar = static_cast<char>(byte.to_ulong());
            file.write(&byteChar, 1);
        }
        file.close();
    }
    else {
        std::cerr << "Unable to open file: " << filename << std::endl;
    }
}

std::string reverseString(const std::string& input) {
    std::string reversed;
    for (auto it = input.rbegin(); it != input.rend(); ++it) {
        reversed += *it;
    }
    return reversed;
}

std::string readBinaryFile(const std::string& filename) {
    std::ifstream file(filename, std::ios::binary);
    if (file.is_open()) {
        std::ostringstream oss;
        char byte;

        while (file.read(&byte, 1)) {
            std::bitset<8> bits(byte);
            oss << bits;
        }

        file.close();
        return oss.str();
    }
    else {
        std::cerr << "Unable to open file: " << filename << std::endl;
        return "";
    }
}

void decodeBinaryFile(const std::string& filename, const std::map<std::string,
char>& decodingMap) {

    std::string encodedBits = readBinaryFile(filename);
    std::string currentCode = "";
    std::string decodedSentence = "";
    for (char bit : encodedBits) {
        std::cout << bit;
        //std::cout << currentCode << std::endl;
        currentCode += bit;
        auto it = decodingMap.find(currentCode);
        if (it != decodingMap.end()) {

            decodedSentence += it->second;
            currentCode.clear();
        }
    }
    std::cout << std::endl;
    std::cout << "Decoded Sentence: " << decodedSentence << std::endl;
}
```

```cpp
int main() {

    setConsoleEncoding();

    int totalCharacters = 0;
    std::map<char, int> charCount = readCharacterCounts("Words.txt",
totalCharacters);

    std::vector<Symbol> probabilityVector;

    calculateValues(charCount, totalCharacters);
    sortTable(charCount, probabilityVector, totalCharacters);
    printTable(probabilityVector, charCount);

    Node* root = nullptr;
    buildShannonFanoTree(probabilityVector, root, 0, probabilityVector.size() -
1, "");

    std::cout << "\nШеннон-Фано дерево:" << std::endl;
    std::cout << "Символ\tКод" << std::endl;

    traverseAndPrintTree(root, "");

    std::map<char, std::string> encodingMap;
    std::map<std::string, char> decodingMap;

    for (const Symbol& element : probabilityVector) {
        encodingMap[element.symbol] = element.code;
    }

    for (const Symbol& element : probabilityVector) {
        decodingMap[element.code] = element.symbol;
    }

    std::string sentence = readSentenceFromFile("Words.txt");
    std::string encodedString = encodeSentence(sentence, encodingMap);
    writeBinaryFile("test.bin", encodedString);

    decodeBinaryFile("test.bin", decodingMap);

    delete root;

    return 0;
}
```